Name: X.Joshua **Reg.no:192212188**
Course: Artificial Intelligence **Code: CSA1709**
Faculty Name: Rajaram p

Experiment No: 1

Title: Write a Python Program to Solve the 8-Puzzle Problem

Aim

To develop a Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm and obtain the sequence of moves from the initial state to the goal state.

Objective

To implement the Breadth First Search (BFS) algorithm for solving the 8-Puzzle Problem by exploring all possible states until the goal state is reached.

Algorithm

- **Step 1:** Define the goal state of the puzzle.
- **Step 2:** Read the initial state from the user.
- **Step 3:** Insert the initial state into the queue.
- **Step 4:** Remove the first state from the queue.
- **Step 5:** Generate all possible valid moves of the blank tile.
- **Step 6:** Compare each new state with the goal state.
- **Step 7:** If the goal state is found, display the solution.
- **Step 8:** Otherwise, add unvisited states to the queue and repeat the process until the goal state is reached.

Program

```
from collections import deque


# Goal State
goal = [1, 2, 3,
    4, 5, 6,
    7, 8, 0]
```

```
# Print the puzzle as a 3x3 matrix
def print_board(state):
    for i in range(0, 9, 3):
        print(state[i], state[i+1], state[i+2])
```

```
    print()

# Generate all possible moves
def get_moves(state):
    moves = []
    pos = state.index(0)

    # Up
    if pos >= 3:
        new = state[:]
        new[pos], new[pos-3] = new[pos-3], new[pos]
        moves.append(new)

    # Down
    if pos <= 5:
        new = state[:]
        new[pos], new[pos+3] = new[pos+3], new[pos]
        moves.append(new)

    # Left
    if pos % 3 != 0:
        new = state[:]
        new[pos], new[pos-1] = new[pos-1], new[pos]
        moves.append(new)

    # Right
    if pos % 3 != 2:
        new = state[:]
        new[pos], new[pos+1] = new[pos+1], new[pos]
```

```
            moves.append(new)

    return moves

# BFS Algorithm
def bfs(start):
    queue = deque([start])
    visited = []

    while queue:
        state = queue.popleft()

        if state == goal:
            print("Goal State Reached!\n")
            print_board(state)
            return

        if state not in visited:
            visited.append(state)

            print("Current State:")
            print_board(state)

            for move in get_moves(state):
                if move not in visited:
                    queue.append(move)

    print("No Solution Found")
```

```
# User Input
print("Enter the Initial State (Use 0 for Blank):")
start = list(map(int, input().split()))

bfs(start)
```

Sample Input

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 6 7 5 8

Sample Output

```
Enter the Initial State (Use 0 for Blank):
1 2 3 4 0 6 7 5 8
Current State:
1 2 3
4 0 6
7 5 8

Current State:
1 0 3
4 2 6
7 5 8

Current State:
1 2 3
4 5 6
7 0 8

Current State:
1 2 3
0 4 6
7 5 8

Current State:
1 2 3
4 6 0
7 5 8

Current State:
0 1 3
4 2 6
7 5 8

Current State:
1 3 0
4 2 6
7 5 8

Current State:
1 2 3
4 5 6
0 7 8

Goal State Reached!

1 2 3
4 5 6
7 8 0
```

Result

Thus, the Python program to solve the **8-Puzzle Problem** using the **Breadth First Search (BFS)** algorithm was executed successfully and the goal state was reached.

Experiment No: 2

Title: Write a Python Program to Solve the 8-Queen Problem

Aim

To develop a Python program to solve the 8-Queen Problem using the Backtracking algorithm by placing eight queens on a chessboard such that no two queens attack each other.

Objective

To implement the Backtracking algorithm for finding a valid arrangement of eight queens on an 8 × 8 chessboard while satisfying all the constraints.

Algorithm

- **Step 1:** Define the size of the chessboard as 8 × 8.
- **Step 2:** Start placing queens from the first row.
- **Step 3:** Check whether the current position is safe.
- **Step 4:** If the position is safe, place the queen.
- **Step 5:** Move to the next row and repeat the process.
- **Step 6:** If no safe position is found, backtrack to the previous row.
- **Step 7:** Continue until all eight queens are placed.
- **Step 8:** Display the final arrangement of queens.

Program

```
N = 8


board = [[0 for _ in range(N)] for _ in range(N)]


def is_safe(row, col):
    # Check left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # Check upper diagonal
    i, j = row, col
```

```
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check lower diagonal
    i, j = row, col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
        return True

    for row in range(N):
        if is_safe(row, col):
            board[row][col] = 1

            if solve(col + 1):
                return True

            board[row][col] = 0
```

```
    return False

if solve(0):
    print("Solution Found:\n")
    for row in board:
        print(*row)
else:
    print("No Solution Exists")
```

Sample Input

No user input required.

Sample Output

```
Solution Found:

1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
```

Result

Thus, the Python program to solve the **8-Queen Problem** using the **Backtracking algorithm** was executed successfully, and a valid arrangement of eight queens was obtained.

Experiment No: 3

Title: Write a Python Program for the Water Jug Problem

Aim

To develop a Python program to solve the Water Jug Problem by determining the sequence of operations required to measure the desired quantity of water using two jugs of different capacities.

Objective

To implement the Water Jug Problem in Python using the operations of filling, emptying, and transferring water between the two jugs until the required quantity of water is obtained.

Algorithm

- **Step 1:** Start the program.
- **Step 2:** Read the capacities of Jug 1, Jug 2, and the target quantity from the user.
- **Step 3:** Check whether the target quantity can be achieved.
- **Step 4:** If the target is not possible, display an appropriate message and stop.
- **Step 5:** Fill Jug 1 with water.
- **Step 6:** Pour water from Jug 1 to Jug 2 and empty Jug 2 whenever it becomes full.
- **Step 7:** Repeat the process until the target quantity is obtained.
- **Step 8:** Display the steps and print the success message.

Program

```
from math import gcd


def water_jug(jug1, jug2, target):
    if target > max(jug1, jug2):
        print("Target cannot be achieved.")
        return

    if target % gcd(jug1, jug2) != 0:
        print("Target cannot be achieved.")
        return
```

```
    x = 0
    y = 0

    print("\nSteps:")

    while x != target and y != target:

        if x == 0:
            x = jug1
            print(f"Fill Jug1       -> ({x}, {y})")

        elif y == jug2:
            y = 0
            print(f"Empty Jug2      -> ({x}, {y})")

        else:
            transfer = min(x, jug2 - y)
            x -= transfer
            y += transfer
            print(f"Pour Jug1 -> Jug2 -> ({x}, {y})")

    print("\nTarget achieved!")

jug1 = int(input("Enter capacity of Jug1: "))
jug2 = int(input("Enter capacity of Jug2: "))
target = int(input("Enter target quantity: "))

water_jug(jug1, jug2, target)
```

Sample Input

Enter capacity of Jug1: 4

Enter capacity of Jug2: 3

Enter target quantity: 2

Sample Output

```
Enter capacity of Jug1: 4
Enter capacity of Jug2: 3
Enter target quantity: 2

Steps:
Fill Jug1       -> (4, 0)
Pour Jug1->Jug2 -> (1, 3)
Empty Jug2      -> (1, 0)
Pour Jug1->Jug2 -> (0, 1)
Fill Jug1       -> (4, 1)
Pour Jug1->Jug2 -> (2, 3)

Target achieved!
```

Result

Thus, the Python program for the **Water Jug Problem** was executed successfully, and the required target quantity of water was obtained.

Experiment No: 4

Title: Write a Python Program for the Crypt-Arithmetic Problem

Aim

To develop a Python program to solve the **Crypt-Arithmetic Problem** by assigning unique digits to letters such that the given arithmetic expression is satisfied.

Objective

To implement the Crypt-Arithmetic problem in Python by assigning unique digits to alphabets and verifying the correctness of the given arithmetic expression.

Algorithm

- **Step 1:** Start the program.
- **Step 2:** Read the words from the user.
- **Step 3:** Identify all unique letters in the words.
- **Step 4:** Generate possible digit assignments for the letters.
- **Step 5:** Assign a unique digit to each letter.
- **Step 6:** Check whether the arithmetic equation is satisfied.
- **Step 7:** If a valid assignment is found, display the solution.
- **Step 8:** Stop the program.

Program

```
from itertools import permutations

# Read input

word1 = input("Enter First Word: ").upper()

word2 = input("Enter Second Word: ").upper()

result = input("Enter Result Word: ").upper()


letters = list(set(word1 + word2 + result))


if len(letters) > 10:

    print("Too many unique letters.")

    exit()
```

```
digits = '0123456789'

found = False

for perm in permutations(digits, len(letters)):
    table = dict(zip(letters, perm))

    # Leading letter cannot be zero
    if table[word1[0]] == '0' or table[word2[0]] == '0' or table[result[0]] == '0':
        continue

    n1 = int(''.join(table[ch] for ch in word1))
    n2 = int(''.join(table[ch] for ch in word2))
    n3 = int(''.join(table[ch] for ch in result))

    if n1 + n2 == n3:
        print("\nSolution Found\n")
        for k in sorted(table):
            print(k, "=", table[k])

        print("\n", n1)
        print("+", n2)
        print(" -----")
        print(n3)
        found = True
        break

if not found:
    print("No Solution Found.")
```

Sample Input

Enter First Word: SEND

Enter Second Word: MORE

Enter Result Word: MONEY

Sample Output

```
Enter First Word: SEND
Enter Second Word: MORE
Enter Result Word: MONEY

Solution Found

D = 7
E = 5
M = 1
N = 6
O = 0
R = 8
S = 9
Y = 2

 9567
+ 1085
------
10652
```

Result

Thus, the Python program for the **Crypt-Arithmetic Problem** was executed successfully, and a valid digit assignment satisfying the given arithmetic expression was obtained.

Experiment No: 5

Title: Write a Python Program for the Missionaries and Cannibals Problem

Aim

To develop a Python program to solve the **Missionaries and Cannibals Problem** by safely transporting all missionaries and cannibals across the river without violating the problem constraints.

Objective

To implement the Missionaries and Cannibals problem in Python by generating valid boat movements and ensuring that the number of cannibals never exceeds the number of missionaries on either riverbank.

Algorithm

- **Step 1:** Start the program.
- **Step 2:** Define the initial state and the goal state.
- **Step 3:** Generate all possible valid boat moves.
- **Step 4:** Check whether each new state is valid.
- **Step 5:** If the state is valid and not visited, add it to the queue.
- **Step 6:** Repeat the process until the goal state is reached.
- **Step 7:** Display the sequence of valid states.
- **Step 8:** Stop the program.

Program

```
from collections import deque


# Check whether the state is valid
def is_valid(m_left, c_left):
    m_right = 3 - m_left
    c_right = 3 - c_left


    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
        return False
    if (m_left > 0 and c_left > m_left):
```

```
    return False

    if (m_right > 0 and c_right > m_right):
        return False

    return True

# BFS Algorithm
def missionaries_cannibals():
    start = (3, 3, 'L')
    goal = (0, 0, 'R')

    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

    queue = deque([[start]])
    visited = set()

    while queue:
        path = queue.popleft()
        state = path[-1]

        if state == goal:
            print("Solution Path:\n")
            for s in path:
                print(s)
            return

        if state in visited:
            continue
```

```
        visited.add(state)

        m, c, boat = state

        for dm, dc in moves:

            if boat == 'L':
                new_state = (m-dm, c-dc, 'R')
            else:
                new_state = (m+dm, c+dc, 'L')

            if is_valid(new_state[0], new_state[1]):
                queue.append(path + [new_state])

    print("No Solution Found")
missionaries_cannibals()
```

Sample Input

No user input required.

Sample Output

```
Solution Path:

(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

Result

Thus, the Python program for the **Missionaries and Cannibals Problem** was executed successfully, and all missionaries and cannibals were safely transported across the river without violating the given constraints.

Experiment No: 6

Title: Write a Python Program for the Vacuum Cleaner Problem

Aim

To develop a Python program to simulate the **Vacuum Cleaner Problem** using a simple reflex agent that cleans all dirty rooms and displays the final room status.

Objective

To implement a Vacuum Cleaner agent in Python that senses the condition of each room and performs the cleaning operation until all rooms become clean.

Algorithm

- **Step 1:** Start the program.
- **Step 2:** Define the rooms and their initial status (Dirty/Clean).
- **Step 3:** Read the status of each room.
- **Step 4:** Check whether the current room is dirty.
- **Step 5:** If the room is dirty, clean the room.
- **Step 6:** Move to the next room and repeat the process.
- **Step 7:** Display the final status of all rooms.
- **Step 8:** Stop the program.

Program

```
# Vacuum Cleaner Problem using DFS

rooms = {}

n = int(input("Enter the number of rooms: "))

for i in range(n):
    room = input(f"Enter Room Name {i+1}: ")
    status = input(f"Enter Status of {room} (Dirty/Clean): ").capitalize()
```

```
    rooms[room] = status

print("\nInitial Room Status")
print(rooms)

visited = set()

def dfs(room):
    if room in visited:
        return

    visited.add(room)

    if rooms[room] == "Dirty":
        print(f"{room} is Dirty -> Cleaning...")
        rooms[room] = "Clean"
    else:
        print(f"{room} is already Clean.")

print("\nCleaning Process")

for room in rooms:
    if room not in visited:
        dfs(room)

print("\nFinal Room Status")
print(rooms)
```

Sample Input

Enter the number of rooms: 2

Enter Room Name 1: A

Enter Status of A (Dirty/Clean): Dirty

Enter Room Name 2: B

Enter Status of B (Dirty/Clean): Dirty

Sample Output

```
Enter the number of rooms: 2
Enter Room Name 1: A
Enter Status of A (Dirty/Clean): Dirty
Enter Room Name 2: B
Enter Status of B (Dirty/Clean): Dirty

Initial Room Status
{'A': 'Dirty', 'B': 'Dirty'}

Cleaning Process
A is Dirty -> Cleaning...
B is Dirty -> Cleaning...

Final Room Status
{'A': 'Clean', 'B': 'Clean'}
```

Result

Thus, the Python program for the **Vacuum Cleaner Problem** was executed successfully, and all dirty rooms were cleaned by the Vacuum Cleaner agent.

Experiment No: 7

Title

Write a Python Program to Implement Breadth First Search (BFS)

Aim

To develop a Python program to implement the **Breadth First Search (BFS)** algorithm for finding the shortest path between a source node and a destination node in a graph.

Objective

To implement the Breadth First Search (BFS) algorithm using a queue to traverse the graph level by level and determine the shortest path between two given nodes.

Algorithm

Step 1: Start the program.

Step 2: Define the graph using an adjacency list.

Step 3: Read the source node and destination node.

Step 4: Initialize a queue with the source node and its path.

Step 5: Create an empty set to store visited nodes.

Step 6: Remove the front node from the queue.

Step 7: If the current node is the destination node, display the shortest path and stop.

Step 8: Otherwise, visit all unvisited neighbouring nodes and insert them into the queue.

Step 9: Repeat Steps 6–8 until the queue becomes empty.

Step 10: If no path exists, display an appropriate message.

Step 11: Stop the program.

Program

```
from collections import deque

# Breadth First Search Function

def bfs(graph, start, goal):
```

```
    queue = deque([(start, [start])])
    visited = set()

    while queue:

        node, path = queue.popleft()

        if node == goal:
            return path

        if node in visited:
            continue

        visited.add(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                queue.append((neighbour, path + [neighbour]))

    return None


# Main Program

graph = {
    0: [1, 2],
    1: [0, 2, 3],
    2: [0, 1, 3],
    3: [1, 2, 4],
```

```
    4: [3]
}

print("Graph Representation")
print("------------------------")

for node in graph:
    print(node, "->", graph[node])

print()

start = int(input("Enter Source Node      : "))
goal = int(input("Enter Destination Node : "))

path = bfs(graph, start, goal)

print()

if path:

    print("Breadth First Search Successful")
    print("-----------------------------------")
    print("Source Node      :", start)
    print("Destination Node :", goal)
    print("Shortest Path    :", " -> ".join(map(str, path)))
    print("Number of Nodes Visited :", len(path))

else:
    print("No Path Found")
```

Sample Input

Enter Source Node 0

Enter Destination Node : 4

Sample Output

```
Graph Representation
--------------------
0 -> [1, 2]
1 -> [0, 2, 3]
2 -> [0, 1, 3]
3 -> [1, 2, 4]
4 -> [3]

Enter Source Node      : 0
Enter Destination Node : 4

Breadth First Search Successful
-------------------------------
Source Node      : 0
Destination Node : 4
Shortest Path    : 0 -> 1 -> 3 -> 4
Number of Nodes Visited : 4
```

Result

Thus, the Python program to implement the **Breadth First Search (BFS)** algorithm was executed successfully, and the shortest path between the source node and destination node was obtained correctly.

Experiment No: 8

Title

Write a Python Program to Implement Depth First Search (DFS)

Aim

To develop a Python program to implement the **Depth First Search (DFS)** algorithm for traversing a graph by visiting each vertex recursively.

Objective

To implement the Depth First Search (DFS) algorithm using recursion to traverse all reachable vertices in a graph from a given starting node.

Algorithm

Step 1: Start the program.

Step 2: Represent the graph using an adjacency list.

Step 3: Create an empty set to store the visited nodes.

Step 4: Select the starting node.

Step 5: Mark the current node as visited and display it.

Step 6: Visit each adjacent unvisited node recursively.

Step 7: Continue the process until all reachable nodes are visited.

Step 8: Stop the program.

Program

```
# Depth First Search (DFS)


graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
```

```
    'E': ['F'],
    'F': []
}

visited = set()

def dfs(node):

    if node not in visited:

        print(node, end=" ")

        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

print("Graph:")
for node in graph:
    print(node, "->", graph[node])

print("\n\nDFS Traversal:")

dfs('A')
```

Sample Input

Starting Node : A

(The graph is predefined in the program.)

Sample Output

```
Graph:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> ['F']
F -> []


DFS Traversal:
A B D E F C
```

Result

Thus, the Python program to implement the **Depth First Search (DFS)** algorithm was executed successfully, and all reachable vertices were traversed recursively.

Experiment No: 9

Title

Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim

To develop a Python program to solve the **Travelling Salesman Problem (TSP)** using the **Nearest Neighbour** approach and determine the shortest tour that visits all cities exactly once and returns to the starting city.

Objective

To implement the Travelling Salesman Problem by calculating the distances between cities and selecting the nearest unvisited city until all cities are visited.

Algorithm

Step 1: Start the program.

Step 2: Read the coordinates of all cities.

Step 3: Construct the distance matrix using Euclidean distance.

Step 4: Select the first city as the starting city.

Step 5: Find the nearest unvisited city.

Step 6: Add the nearest city to the tour and update the total distance.

Step 7: Repeat Steps 5 and 6 until all cities are visited.

Step 8: Return to the starting city.

Step 9: Display the tour and total distance.

Step 10: Stop the program.

Program

```
import numpy as np


def tsp(cities):
```

```
n = len(cities)

# Create Distance Matrix
distance = np.zeros((n, n))

for i in range(n):
    for j in range(n):
        if i != j:
            distance[i][j] = np.linalg.norm(cities[i] - cities[j])

visited = [False] * n
visited[0] = True

tour = [0]
total_distance = 0
current = 0

while len(tour) < n:

    nearest = -1
    minimum = float('inf')

    for city in range(n):

        if not visited[city] and distance[current][city] < minimum:
            minimum = distance[current][city]
            nearest = city

    tour.append(nearest)
```

```
            visited[nearest] = True
            total_distance += minimum
            current = nearest

        total_distance += distance[current][0]
        tour.append(0)

        return tour, total_distance


# Main Program

n = int(input("Enter Number of Cities: "))

cities = []

for i in range(n):
    x = float(input(f"Enter x-coordinate of City {i}: "))
    y = float(input(f"Enter y-coordinate of City {i}: "))
    cities.append(np.array([x, y]))

tour, distance = tsp(cities)

print("\nOptimal Tour")
print(" -> ".join(map(str, tour)))

print("Total Distance =", round(distance,2))
```

Sample Input

Enter Number of Cities: 4

City 0 : (0,0)

City 1 : (2,3)

City 2 : (5,2)

City 3 : (6,6)

Sample Output

```
Enter Number of Cities: 4
Enter x-coordinate of City 0: 0
Enter y-coordinate of City 0: 0
Enter x-coordinate of City 1: 2
Enter y-coordinate of City 1: 3
Enter x-coordinate of City 2: 5
Enter y-coordinate of City 2: 2
Enter x-coordinate of City 3: 6
Enter y-coordinate of City 3: 6

Optimal Tour
0 -> 1 -> 2 -> 3 -> 0
Total Distance = 19.38
```

Result: Thus the Program was executed and the output was found to be Correct.

Experiment No: 10

Title

Write a Python Program to Implement A* Algorithm

Aim

To develop a Python program to implement the **A*** search algorithm for finding the shortest path between a start node and a goal node.

Objective

To implement the A* search algorithm using a priority queue and heuristic function to determine the optimal path from the source node to the destination node.

Algorithm

Step 1: Start the program.

Step 2: Define the graph and heuristic values.

Step 3: Initialize the open list with the start node.

Step 4: Select the node with the lowest **f(n) = g(n) + h(n)**.

Step 5: Expand the current node and calculate the cost of its neighbours.

Step 6: Update the cost and parent of each neighbour if a shorter path is found.

Step 7: Repeat until the goal node is reached or the open list becomes empty.

Step 8: Display the shortest path and total cost.

Step 9: Stop the program.

Program

```
import heapq


graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
```

```
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 3)],
    'G': []
}

heuristic = {
    'A': 8,
    'B': 6,
    'C': 5,
    'D': 3,
    'E': 2,
    'F': 2,
    'G': 0
}

def astar(start, goal):

    priority_queue = []
    heapq.heappush(priority_queue, (0, start))

    parent = {}
    cost = {}

    parent[start] = None
    cost[start] = 0

    while priority_queue:
```

```
    current = heapq.heappop(priority_queue)[1]

    if current == goal:
        break

    for neighbour, weight in graph[current]:

        new_cost = cost[current] + weight

        if neighbour not in cost or new_cost < cost[neighbour]:

            cost[neighbour] = new_cost

            priority = new_cost + heuristic[neighbour]

            heapq.heappush(priority_queue, (priority, neighbour))

            parent[neighbour] = current

path = []

node = goal

while node is not None:
    path.append(node)
    node = parent[node]

path.reverse()
```

```
    return path, cost[goal]



start = input("Enter Start Node : ").upper()

goal = input("Enter Goal Node  : ").upper()


path, total_cost = astar(start, goal)


print("\nShortest Path :", " -> ".join(path))

print("Total Cost    :", total_cost)
```

Sample Input

Enter Start Node : A

Enter Goal Node : G

Sample Output

```
Enter Start Node : A
Enter Goal Node  : G

Shortest Path : A -> B -> D -> G
Total Cost    : 9
```

Result

Thus, the Python program to implement the **A*** algorithm was executed successfully, and the shortest path along with the minimum cost was obtained correctly.

Experiment No: 11

Title

Write a Python Program for Map Coloring using Constraint Satisfaction Problem (CSP)

Aim

To develop a Python program to solve the **Map Coloring Problem** using the **Constraint Satisfaction Problem (CSP)** approach by assigning different colors to adjacent regions.

Objective

To implement the Map Coloring problem using the backtracking technique of CSP so that no two neighbouring regions are assigned the same color.

Algorithm

Step 1: Start the program.

Step 2: Define the map regions and available colors.

Step 3: Specify the neighbouring regions as constraints.

Step 4: Assign a color to each region one by one.

Step 5: Check whether the assigned color satisfies all constraints.

Step 6: If valid, continue assigning colors to the next region.

Step 7: If no valid color is available, backtrack and try another color.

Step 8: Repeat until all regions are colored.

Step 9: Display the final color assignment.

Step 10: Stop the program.

Program

```
# Map Coloring using CSP (Backtracking)


regions = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
```

```
colors = ['Red', 'Green', 'Blue']

neighbours = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['SA', 'Q', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

assignment = {}

def is_safe(region, color):

    for neighbour in neighbours[region]:
        if neighbour in assignment and assignment[neighbour] == color:
            return False

    return True



def solve(index):

    if index == len(regions):
        return True

    region = regions[index]
```

```
    for color in colors:

        if is_safe(region, color):

            assignment[region] = color

            if solve(index + 1):
                return True

            del assignment[region]

    return False



if solve(0):

    print("\nColor Assignment\n")

    for region in regions:
        print(region, ":", assignment[region])

else:

    print("No Solution Found")
```

Sample Input

No user input.

Sample Output

```
Color Assignment

WA : Red
NT : Green
SA : Blue
Q : Red
NSW : Green
V : Red
T : Red
```

Result

Thus, the Python program for **Map Coloring using Constraint Satisfaction Problem (CSP)** was executed successfully, and the map was colored without assigning the same color to any adjacent regions.

Experiment No: 12

Title

Write a Python Program for Tic Tac Toe Game

Aim

To develop a Python program to implement the **Tic Tac Toe** game for two players and determine the winner or draw based on the game rules.

Objective

To implement a two-player Tic Tac Toe game using Python and verify the winning conditions after each move.

Algorithm

Step 1: Start the program.

Step 2: Initialize a 3 × 3 game board with empty spaces.

Step 3: Display the board.

Step 4: Allow Player X and Player O to enter their moves alternately.

Step 5: Check whether the entered position is empty.

Step 6: After every move, check for a winning condition.

Step 7: If a player wins, display the winner.

Step 8: If all positions are filled without a winner, declare a draw.

Step 9: Stop the program.

Program

```
# Tic Tac Toe Game


board = [' ' for _ in range(9)]


def display_board():
    print()
```

```
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()

def check_win(player):

    win = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]

    for x, y, z in win:
        if board[x] == board[y] == board[z] == player:
            return True

    return False



def board_full():
    return ' ' not in board



player = 'X'

while True:
```

```
display_board()

position = int(input(f"Player {player}, Enter Position (1-9): ")) - 1

if position < 0 or position > 8:
    print("Invalid Position")
    continue

if board[position] != ' ':
    print("Position Already Filled")
    continue

board[position] = player

if check_win(player):
    display_board()
    print("Player", player, "Wins!")
    break

if board_full():
    display_board()
    print("Match Draw")
    break

if player == 'X':
    player = 'O'
else:
    player = 'X'
```

Sample Input

Player X, Enter Position (1-9): 1

Player O, Enter Position (1-9): 5

Player X, Enter Position (1-9): 2

Player O, Enter Position (1-9): 8

Player X, Enter Position (1-9): 3

Sample Output

```
  |   |
--+---+--
  |   |
--+---+--
  |   |

Player X, Enter Position (1-9): 1

X |   |
--+---+--
  |   |
--+---+--
  |   |

Player O, Enter Position (1-9): 5

X |   |
--+---+--
  | O |
--+---+--
  |   |

Player X, Enter Position (1-9): 2

X | X |
--+---+--
  | O |
--+---+--
  |   |

Player O, Enter Position (1-9): 8

X | X |
--+---+--
  | O |
--+---+--
  | O |

Player X, Enter Position (1-9): 3

X | X | X
--+---+--
  | O |
--+---+--
  | O |

Play   X Wins!
>>>
```

Result

Thus, the Python program for the **Tic Tac Toe Game** was executed successfully, and the winner was determined based on the game rules.

Experiment No: 13

Title

Write a Python Program to Implement Minimax Algorithm for Gaming

Aim

To develop a Python program to implement the **Minimax Algorithm** for selecting the optimal move in a two-player game.

Objective

To implement the Minimax algorithm to evaluate game states and determine the best possible move for a player.

Algorithm

Step 1: Start the program.

Step 2: Define the game tree values.

Step 3: Check whether the current node is a leaf node.

Step 4: If it is the maximizing player's turn, select the maximum value.

Step 5: If it is the minimizing player's turn, select the minimum value.

Step 6: Repeat recursively until the root node is evaluated.

Step 7: Display the optimal value.

Step 8: Stop the program.

Program

```
# Minimax Algorithm


scores = [3, 5, 2, 9, 12, 5, 23, 23]


def minimax(depth, node, maximizing):


    if depth == 3:
```

```
        return scores[node]

    if maximizing:

        left = minimax(depth + 1, node * 2, False)
        right = minimax(depth + 1, node * 2 + 1, False)

        return max(left, right)

    else:

        left = minimax(depth + 1, node * 2, True)
        right = minimax(depth + 1, node * 2 + 1, True)

        return min(left, right)



result = minimax(0, 0, True)

print("Optimal Value :", result)
```

Sample Input

No user input.

Sample Output

```
------------------------------
Optimal Value : 12
```

Result

Thus, the Python program to implement the **Minimax Algorithm** was executed successfully, and the optimal game value was obtained correctly.

Experiment No: 14

Title

Write a Python Program to Implement Alpha-Beta Pruning Algorithm for Gaming

Aim

To develop a Python program to implement the **Alpha-Beta Pruning** algorithm for finding the optimal move in a game.

Objective

To implement the Alpha-Beta Pruning algorithm for reducing unnecessary node evaluation while determining the optimal game value.

Algorithm

Step 1: Start the program.

Step 2: Define the leaf node values.

Step 3: Initialize alpha as $-\infty$ and beta as $+\infty$.

Step 4: Traverse the game tree recursively.

Step 5: If it is the maximizing player's turn, choose the maximum value and update alpha.

Step 6: If it is the minimizing player's turn, choose the minimum value and update beta.

Step 7: If **alpha ≥ beta**, prune the remaining branches.

Step 8: Display the optimal value.

Step 9: Stop the program.

Program

```
# Alpha-Beta Pruning Algorithm


scores = [3, 5, 2, 9, 12, 5, 23, 23]


def alphabeta(depth, node, alpha, beta, maximizing):
```

```
if depth == 3:
    return scores[node]

if maximizing:

    value = float('-inf')

    for i in range(2):

        value = max(value,
                    alphabeta(depth + 1,
                              node * 2 + i,
                              alpha,
                              beta,
                              False))

        alpha = max(alpha, value)

        if alpha >= beta:
            break

    return value

else:

    value = float('inf')

    for i in range(2):
```

```
        value = min(value,
                alphabeta(depth + 1,
                        node * 2 + i,
                        alpha,
                        beta,
                        True))

        beta = min(beta, value)

        if alpha >= beta:
            break

    return value


result = alphabeta(0, 0, float('-inf'), float('inf'), True)

print("Optimal Value :", result)
```

Sample Input

No user input.

Sample Output

```
------------------------------
Optimal Value : 12
```

Result

Thus, the Python program to implement the **Alpha-Beta Pruning Algorithm** was executed successfully, and the optimal game value was obtained by pruning unnecessary branches.

Experiment No: 15

Title

Write a Python Program to Implement Decision Tree

Aim

To develop a Python program to implement a **Decision Tree Classifier** for classifying data and predicting the output.

Objective

To build and test a Decision Tree model using the Iris dataset and evaluate its prediction accuracy.

Algorithm

Step 1: Import the required libraries.

Step 2: Load the Iris dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Create a Decision Tree classifier.

Step 5: Train the classifier using the training data.

Step 6: Predict the output for the testing data.

Step 7: Calculate and display the accuracy.

Step 8: Stop the program.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()
```

```
X = iris.data
y = iris.target

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42
)

# Create Decision Tree Model
model = DecisionTreeClassifier(random_state=42)

# Train Model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Predicted Classes:")
print(y_pred)

print("\nAccuracy:", round(accuracy * 100, 2), "%")
```

Sample Input

No user input.

(Iris dataset is loaded automatically.)

Sample Output

```
Predicted Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy: 100.0 %
>>>
```

Result

Thus, the Python program to implement the **Decision Tree Classifier** was executed successfully, and the classification accuracy was obtained correctly.

Experiment 16

Title

Write the python program to implement Feed forward neural Network

Aim

To develop a Python program to implement a Feed Forward Neural Network for classifying the Iris dataset.

Objective

To train a neural network using the Iris dataset and evaluate its classification accuracy.

Algorithm

Step 1: Import the required libraries.

Step 2: Load the Iris dataset.

Step 3: Split the dataset into training and testing sets.

Step 4: Create a Feed Forward Neural Network with one hidden layer and one output layer.

Step 5: Compile the model using the Adam optimizer and categorical crossentropy loss function.

Step 6: Train the model using the training dataset.

Step 7: Test the model using the testing dataset.

Step 8: Display the classification accuracy.

Program

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

from tensorflow.keras.utils import to_categorical

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split


# Load Iris Dataset
```

```
iris = load_iris()

X = iris.data
y = iris.target

# Convert target values into one-hot encoding
y = to_categorical(y)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create Feed Forward Neural Network
model = Sequential()

# Hidden Layer
model.add(Dense(10, input_shape=(4,), activation='relu'))

# Output Layer
model.add(Dense(3, activation='softmax'))

# Compile Model
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

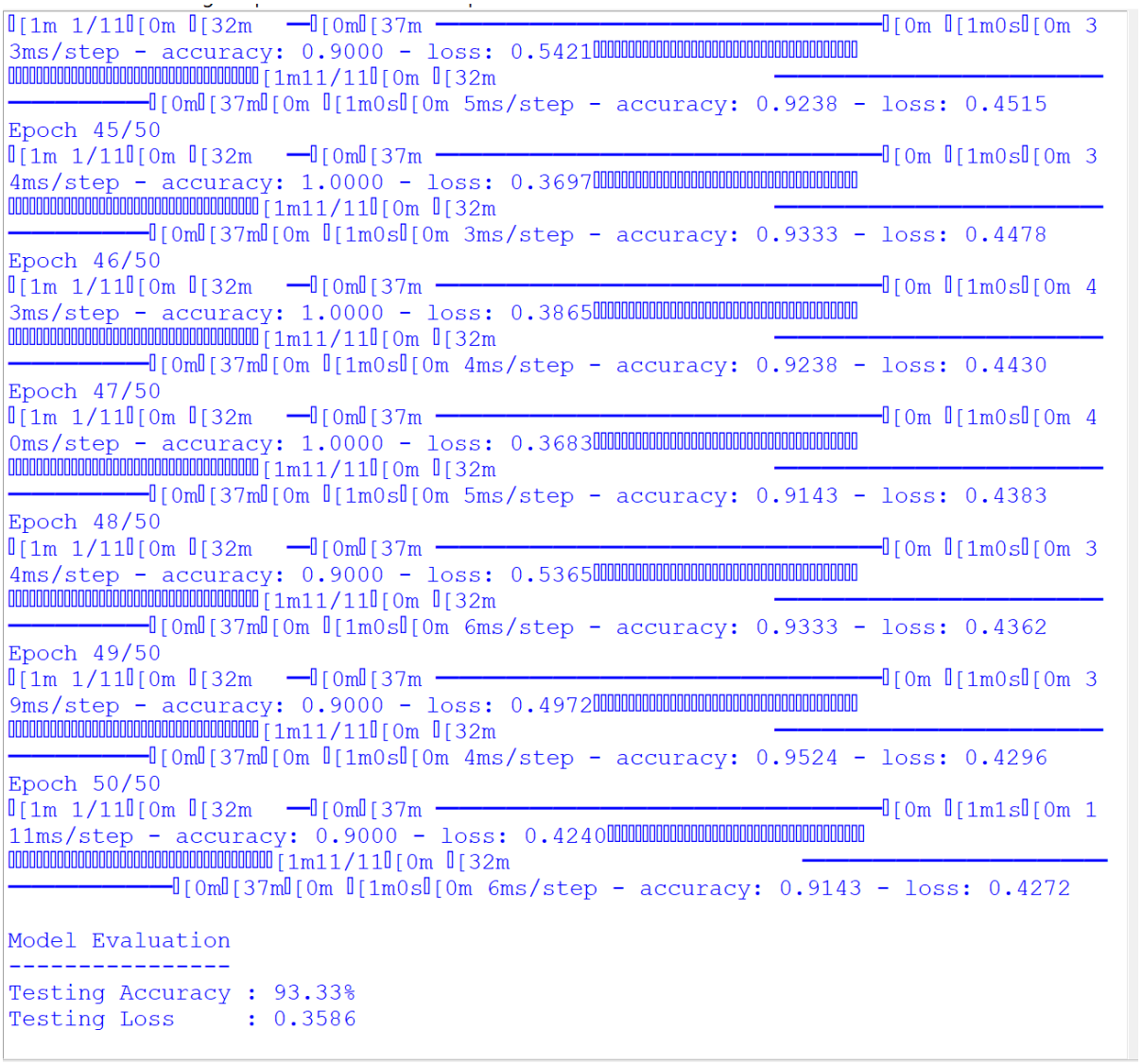
```
# Train Model
model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=10,
    verbose=1
)

# Evaluate Model
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("\nModel Evaluation")
print("------------------")
print("Testing Accuracy : {:.2f}%".format(accuracy * 100))
print("Testing Loss     : {:.4f}".format(loss))
```

Sample Input

No manual input is required.

The Iris dataset is loaded automatically by the program.

Sample Output



Result

Thus, the Python program to implement a **Feed Forward Neural Network** was executed successfully, and the classification accuracy of the Iris dataset was obtained.

Experiment 17

Title

Write a Prolog Program to Sum the Integers from 1 to n.

Aim

To develop a Prolog program to find the sum of integers from **1 to n** using recursion.

Objective

To understand the concept of recursion in Prolog by computing the sum of natural numbers from **1 to n**, using a base case and a recursive rule.

Algorithm

Step 1: Define the base case where the sum of 0 is 0.

Step 2: Accept a positive integer **N**.

Step 3: Decrease **N** by 1.

Step 4: Recursively calculate the sum from **1** to **N−1**.

Step 5: Add **N** to the previous sum.

Step 6: Display the final sum.

Program

```
% Base Case
sum(0,0).

% Recursive Case
sum(N,Sum) :-
    N > 0,
    N1 is N - 1,
    sum(N1,Sum1),
    Sum is Sum1 + N.
```

Sample Input

?- sum(5,Sum).

Sample Output

Result

Thus, the Prolog program to find the **sum of integers from 1 to n** using recursion was executed successfully, and the correct sum was obtained.

Experiment 18

Title

Prolog Program for a Database with Name and Date of Birth (DOB)

Aim

To develop a Prolog program to create a simple database containing a person's name and date of birth and perform database queries.

Objective

To understand how facts and predicates are used in Prolog to store and retrieve information from a simple database.

Algorithm

Step 1: Define facts containing the person's name and date of birth.

Step 2: Create a predicate to retrieve the DOB using the person's name.

Step 3: Enter the query with the required name.

Step 4: Search the database for the matching record.

Step 5: Display the corresponding date of birth.

Program

```
% Database of Names and Date of Birth

person(john, date(1990, 5, 1)).
person(jane, date(1985, 12, 10)).
person(bob,  date(1978, 2, 28)).
person(sue,  date(1995, 8, 15)).
person(tom,  date(2000, 4, 22)).

% Predicate to retrieve DOB
find_dob(Name, DOB) :-
```

person(Name, DOB).

Sample Input

?- find_dob(john, DOB).

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp18.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp18.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp18.pl compiled, 11 lines read - 1236 bytes written, 6 ms

yes
| ?- find_dob(john, DOB).

DOB = date(1990,5,1)

yes
| ?- |
```

Result

Thus, the Prolog program to create a simple **Name and Date of Birth database** was executed successfully, and the required records were retrieved correctly.

Experiment 19

Title

Write a Prolog Program for Student–Teacher–Subject Database

Aim

To implement a Student–Teacher–Subject database using Prolog facts and retrieve teacher and student information through queries.

Objective

To understand how Prolog stores facts and uses rules to search and retrieve information from a simple database.

Algorithm

Step 1: Define facts for teachers and the subjects they teach.

Step 2: Define facts for students and the subjects they study.

Step 3: Create a rule to retrieve the subject handled by a teacher.

Step 4: Create another rule to retrieve the students studying a particular subject.

Step 5: Execute the required query and display the matching result.

Program

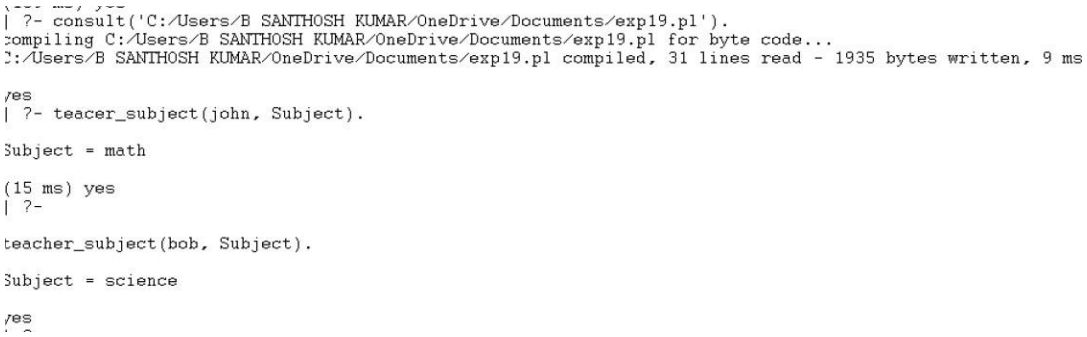
```
% Teacher and Subject Database

teacher(john, math).
teacher(jane, english).
teacher(bob, science).
teacher(sue, history).
teacher(tom, art).

% Student and Subject Database

student(alice, math).
student(alice, science).
```

```
student(bob, english).
student(bob, science).

student(carol, history).
student(carol, art).

student(dave, math).
student(dave, english).
student(dave, art).

% Retrieve teacher subject

teacher_subject(Teacher, Subject) :-
    teacher(Teacher, Subject).

% Retrieve students studying a subject

subject_student(Subject, Student) :-
    student(Student, Subject).
```

Sample Input

```
?- teacher_subject(john, Subject).
```

Sample Output



Result

Hence, the Prolog program for the Student–Teacher–Subject database was executed successfully and the correct output was obtained.

Experiment 20

Title

Write a Prolog Program for Planets Database

Aim

To implement a Planets database using Prolog facts and retrieve the properties of a planet using queries.

Objective

To understand how Prolog stores planetary information and displays the required details based on the given planet name.

Algorithm

Step 1: Define facts for each planet with its type, size, temperature, and position.

Step 2: Create a rule to retrieve the details of a selected planet.

Step 3: Accept the planet name through a query.

Step 4: Search the database for the matching planet.

Step 5: Display the planet's properties such as type, size, temperature, and position.

Program

```
% Planet Database

planet(mercury, rocky, small, hot, first).
planet(venus, rocky, small, hot, second).
planet(earth, rocky, medium, moderate, third).
planet(mars, rocky, small, cold, fourth).
planet(jupiter, gas_giant, large, cold, fifth).
planet(saturn, gas_giant, large, cold, sixth).
planet(uranus, ice_giant, large, cold, seventh).
planet(neptune, ice_giant, large, cold, eighth).

% Display Planet Information
```

```
show_planet(Name) :-
    planet(Name, Type, Size, Temperature, Position),
    write('Planet Name : '), write(Name), nl,
    write('Type        : '), write(Type), nl,
    write('Size        : '), write(Size), nl,
    write('Temperature : '), write(Temperature), nl,
    write('Position    : '), write(Position), nl.
```

Sample Input

?- show_planet(earth).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp20.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp20.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp20.pl compiled, 20 lines read - 2866 bytes written, 7 ms

(16 ms) yes
| ?- show_planet(earth).
Planet Name : earth
Type        : rocky
Size        : medium
Temperature : moderate
Position    : third
```

Result

Hence, the Prolog program for the Planets Database was executed successfully and the correct output was obtained.

Experiment 21

Title

Write a Prolog Program to implement Towers of Hanoi

Aim

To implement the Towers of Hanoi problem using Prolog and display the sequence of moves required to transfer all the disks from the source peg to the destination peg.

Objective

To understand the concept of recursion in Prolog by solving the Towers of Hanoi problem and generating the correct sequence of disk movements.

Algorithm

Step 1: Define a base case to move one disk from the source peg to the destination peg.

Step 2: If the number of disks is greater than one, move **N−1** disks from the source peg to the auxiliary peg.

Step 3: Move the largest disk from the source peg to the destination peg.

Step 4: Move the **N−1** disks from the auxiliary peg to the destination peg.

Step 5: Display each move until all the disks are transferred successfully.

Program

```
% Towers of Hanoi


hanoi(1, Source, _, Destination) :-
    write('Move disk 1 from '),
    write(Source),
    write(' to '),
    write(Destination), nl.


hanoi(N, Source, Auxiliary, Destination) :-
    N > 1,
    M is N - 1,
    hanoi(M, Source, Destination, Auxiliary),
    write('Move disk '),
```

```
    write(N),
    write(' from '),
    write(Source),
    write(' to '),
    write(Destination),  nl,
    hanoi(M, Auxiliary, Source, Destination).
```

Sample Input

?- hanoi(3, 'A', 'B', 'C').

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp21.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp21.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp21.pl compiled, 18 lines read - 1718 bytes written, 4 ms

(31 ms) yes
| ?- hanoi(3, 'A', 'B', 'C').
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

true ?

(31 ms) yes
```

Result

Hence, the Prolog program to implement the **Towers of Hanoi** was executed successfully and the correct sequence of moves was obtained.

Experiment 22

Title

Write a Prolog Program to print whether a particular bird can fly or not.

Aim

To implement a Prolog program to determine whether a particular bird can fly using facts and queries.

Objective

To understand the use of facts and rules in Prolog by identifying whether a given bird can fly or not.

Algorithm

Step 1: Define facts for different birds and their flying ability.

Step 2: Create a rule to check whether a bird can fly.

Step 3: Enter the bird name through a query.

Step 4: Prolog searches the database for the given bird.

Step 5: Display whether the bird can fly or not.

Program

```
% Bird Database

bird(ostrich, cannot_fly).
bird(penguin, cannot_fly).
bird(kiwi, cannot_fly).
bird(eagle, can_fly).
bird(pigeon, can_fly).
bird(sparrow, can_fly).

% Rule to check flying ability

can_fly(Bird) :-
    bird(Bird, can_fly).
```

cannot_fly(Bird) :-

bird(Bird, cannot_fly).

Sample Input

?- can_fly(eagle).

?- can_fly(ostrich).

?- bird(Bird, cannot_fly).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp22.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp22.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp22.pl compiled, 15 lines read - 1131 bytes written, 3 ms

(15 ms) yes
| ?- can_fly(eagle).

yes
| ?- can_fly(ostrich).

no
| ?- bird(Bird, cannot_fly).

Bird = ostrich ?

yes
| ?-
bird(Bird, cannot_fly).

Bird = ostrich ? ;

Bird = penguin ? ;

Bird = kiwi ? ;

(78 ms) no
| ?-
```

Result

Hence, the Prolog program to determine whether a particular bird can fly or not was executed successfully and the correct output was obtained.

Experiment 23

Title

Write a Prolog Program to implement a Family Tree.

Aim

To implement a Family Tree in Prolog using facts and rules to identify different family relationships.

Objective

To understand how Prolog uses facts and rules to represent and retrieve family relationships such as **mother, father, grandfather, grandmother, sister, and brother**.

Algorithm

Step 1: Define facts for male and female family members.

Step 2: Define parent relationships between the family members.

Step 3: Create rules for **mother** and **father** using gender and parent facts.

Step 4: Create rules for **grandfather**, **grandmother**, **sister**, and **brother**.

Step 5: Execute queries to retrieve the required family relationships.

Program

```
% Family Members

female(pam).
female(liz).
female(ann).
female(pat).

male(tom).
male(bob).
male(jim).

% Parent Relations

parent(tom, liz).
```

```
parent(tom, bob).
parent(pam, liz).
parent(pam, bob).
parent(bob, ann).
parent(bob, pat).
parent(liz, jim).

% Mother

mother(X, Y) :-
    female(X),
    parent(X, Y).

% Father

father(X, Y) :-
    male(X),
    parent(X, Y).

% Grandmother

grandmother(X, Y) :-
    female(X),
    parent(X, Z),
    parent(Z, Y).

% Grandfather

grandfather(X, Y) :-
```

```
    male(X),
    parent(X, Z),
    parent(Z, Y).

% Sister

sister(X, Y) :-
    female(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.

% Brother

brother(X, Y) :-
    male(X),
    parent(Z, X),
    parent(Z, Y),
    X \= Y.
```

Sample Input

```
?- father(X, bob).
?- mother(X, bob).
?- grandfather(X, ann).
?- grandmother(X, ann).
?- sister(X, pat).
?- brother(X, liz).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp23.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp23.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp23.pl compiled, 61 lines read - 3759 bytes written, 5 ms

(16 ms) yes
| ?- father(X, bob).

X = tom ?

(109 ms) yes
| ?- mother(X, bob).

X = pam ?

(63 ms) yes
| ?- grandfather(X, ann).

X = tom ?

yes
| ?- grandmother(X, ann).

X = pam ?

(47 ms) yes
| ?- sister(X, pat).

X = ann ?

yes
| ?- brother(X, liz).

X = bob ?

(47 ms) yes
| ?-
```

Result

Hence, the Prolog program to implement a **Family Tree** was executed successfully and the correct family relationships were obtained.

Experiment 24

Title

Write a Prolog Program to suggest a Dieting System based on Disease.

Aim

To implement a Prolog program that suggests a suitable diet based on a given disease.

Objective

To understand how Prolog uses facts and rules to recommend an appropriate diet for different diseases.

Algorithm

Step 1: Define facts for different diseases and their recommended diets.

Step 2: Create a rule to retrieve the diet for a given disease.

Step 3: Enter the disease name through a query.

Step 4: Prolog searches the database for the matching disease.

Step 5: Display the recommended diet.

Program

```
% Diet Database

diet(heart_disease, [apple, carrot, chicken, fish, almonds]).

diet(diabetes, [apple, carrot, fish, spinach, almonds]).

diet(anemia, [spinach, chicken, fish, almonds]).

diet(obesity, [apple, carrot, fish, spinach]).

% Rule to Suggest Diet

suggest_diet(Disease, Diet) :-
    diet(Disease, Diet).
```

Sample Input

?- suggest_diet(diabetes, Diet).

?- suggest_diet(obesity, Diet).

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp24.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp24.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp24.pl compiled, 10 lines read - 1632 bytes written, 5 ms

(16 ms) yes
| ?- suggest_diet(diabetes, Diet).

Diet = [apple,carrot,fish,spinach,almonds]

(16 ms) yes
| ?- suggest_diet(obesity, Diet).

Diet = [apple,carrot,fish,spinach]

yes
```

Result

Hence, the Prolog program to suggest a **Dieting System based on Disease** was executed successfully and the correct diet recommendations were obtained.

Experiment 25

Title

Write a Prolog Program to implement the Monkey Banana Problem.

Aim

To implement the Monkey Banana Problem using Prolog and determine the sequence of actions required for the monkey to obtain the banana.

Objective

To understand how Prolog uses facts and rules to solve a state-space problem by finding the required sequence of actions.

Algorithm

Step 1: Define the initial state of the monkey, box, and banana.

Step 2: Define the final state where the monkey has obtained the banana.

Step 3: Define the possible actions and their effects on the current state.

Step 4: Execute the actions recursively until the goal state is reached.

Step 5: Display the sequence of actions required to solve the problem.

Program

```
% Initial and Final States

initial_state(state(at_door, on_floor, at_window, has_not_eaten)).
final_state(state(_, _, _, has_eaten)).

% Actions

action(state(at_door, on_floor, at_window, has_not_eaten),
    climb,
    state(at_window, on_window, at_window, has_not_eaten)).

action(state(at_window, on_window, at_window, has_not_eaten),
    grasp,
```

```
    state(at_window, on_window, at_window, has_eaten)).

% Execute Actions

execute([], State, State).

execute([Action|Rest], CurrentState, FinalState) :-
    action(CurrentState, Action, NextState),
    execute(Rest, NextState, FinalState).

% Solve Problem

solve(Actions) :-
    initial_state(State),
    execute(Actions, State, FinalState),
    final_state(FinalState).
```

Sample Input

```
?- solve(Actions)
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp25.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp25.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp25.pl compiled, 28 lines read - 2564 bytes written, 4 ms

yes
| ?- solve(Actions).

Actions = [climb,grasp] ?

yes
```

Result

Hence, the Prolog program to implement the **Monkey Banana Problem** was executed successfully and the correct sequence of actions was obtained.

Experiment 26

Title

Write a Prolog Program for Fruit and its Color using Backtracking.

Aim

To implement a Prolog program that identifies the color of a fruit using backtracking.

Objective

To understand how Prolog uses facts and backtracking to retrieve fruits and their corresponding colors.

Algorithm

Step 1: Define facts representing fruits and their colors.

Step 2: Create a rule to match a fruit with its color.

Step 3: Enter the fruit name or color through a query.

Step 4: Prolog searches the database for matching facts.

Step 5: Using backtracking, Prolog displays all possible matching fruits and colors.

Program

```
% Fruit Database

fruit(apple, red).
fruit(banana, yellow).
fruit(grape, purple).
fruit(orange, orange).
fruit(watermelon, green).

% Rule to Match Fruit and Color

fruit_color(Fruit, Color) :-
    fruit(Fruit, Color).
```

Sample Input

?- fruit_color(Fruit, Color).

Sample Output

```
| ?- consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp26.pl compiled, 11 lines read - 826 bytes written, 7 ms

yes
| ?- fruit_color(Fruit, Color).

Color = red
Fruit = apple ? ;

Color = yellow
Fruit = banana ? ;

Color = purple
Fruit = grape ? ;

Color = orange
Fruit = orange ? ;

Color = green
Fruit = watermelon

(109 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Fruit and its Color using Backtracking** was executed successfully and the correct fruit-color combinations were obtained.

Experiment 27

Title

Write a Prolog Program to implement Best First Search Algorithm.

Aim

To implement the Best First Search algorithm in Prolog to find the best path from the source node to the goal node.

Objective

To understand how Best First Search selects the most promising node based on the minimum edge cost and finds a path to the goal.

Algorithm

Step 1: Define the graph using facts with edge costs.

Step 2: Specify the source node and the goal node.

Step 3: Select the adjacent node having the lowest cost.

Step 4: Continue the search while avoiding already visited nodes.

Step 5: Display the best path from the source node to the goal node.

Program

```
% Graph with Edge Costs

edge(a,b,2).
edge(a,c,5).
edge(b,d,3).
edge(b,e,6).
edge(c,f,7).
edge(d,g,2).
edge(e,g,4).
edge(f,g,1).

% Best First Search
```

```
best_first(Start, Goal, Path) :-
    bfs(Start, Goal, [Start], RevPath),
    reverse(RevPath, Path).

bfs(Goal, Goal, Visited, Visited).

bfs(Current, Goal, Visited, Path) :-
    findall(Cost-Next,
        (edge(Current, Next, Cost),
         \+ member(Next, Visited)),
        Neighbours),
    sort(Neighbours, [_-Best|_]),
    bfs(Best, Goal, [Best|Visited], Path).
```

Sample Input

```
?- best_first(a,g,Path).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp27.pl compiled, 25 lines read - 2534 bytes written, 8 ms

yes
| ?- best_first(a,g,Path).

Path = [a,b,d,g] ? ;

(15 ms) no
| ?-

best_first(a,g,Path).

Path = [a,b,d,g] ?

(31 ms) yes
| ?-
```

Result

Hence, the Prolog program to implement the **Best First Search Algorithm** was executed successfully and the best path was obtained.

Experiment 28

Title

Write a Prolog Program for Medical Diagnosis.

Aim

To implement a Prolog program for medical diagnosis based on the symptoms of a patient.

Objective

To understand how Prolog uses facts and rules to diagnose diseases based on the symptoms provided.

Algorithm

Step 1: Define the symptoms of different patients as facts.

Step 2: Define rules that relate symptoms to diseases.

Step 3: Enter the patient name through a query.

Step 4: Prolog searches for the patient's symptoms.

Step 5: Display the diagnosed disease if the symptoms match a disease rule.

Program

```
% Symptoms of Patients

symptom(john, fever).
symptom(john, cough).
symptom(john, headache).

symptom(anu, fever).
symptom(anu, rash).

symptom(rahul, fever).
symptom(rahul, body_ache).
symptom(rahul, cough).
```

```
% Disease Rules

disease(Patient, flu) :-
    symptom(Patient, fever),
    symptom(Patient, cough),
    symptom(Patient, headache).

disease(Patient, measles) :-
    symptom(Patient, fever),
    symptom(Patient, rash).

disease(Patient, viral_fever) :-
    symptom(Patient, fever),
    symptom(Patient, body_ache),
    symptom(Patient, cough).

% Diagnosis

diagnose(Patient) :-
    disease(Patient, Disease),
    write('Disease : '),
    write(Disease), nl.
```

Sample Input

```
?- diagnose(john).

?- diagnose(anu).
```

?- diagnose(rahul).

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp28.pl compiled, 34 lines read - 2468 bytes written, 9 ms

yes
| ?- diagnose(john).
Disease : flu

true ?

(78 ms) yes
| ?- diagnose(anu).
Disease : measles

true ?

(47 ms) yes
| ?- diagnose(rahul).
Disease : viral_fever

true ?

(78 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Medical Diagnosis** was executed successfully and the correct disease was diagnosed.

Experiment 29

Title

Write a Prolog Program for Forward Chaining. Incorporate Required Queries.

Aim

To implement Forward Chaining in Prolog to infer new facts from the given knowledge base.

Objective

To understand how Prolog derives new facts by applying rules to the available facts in the knowledge base.

Algorithm

Step 1: Define the initial facts in the knowledge base.

Step 2: Define inference rules to derive new facts.

Step 3: Execute the forward chaining rule.

Step 4: Prolog applies the rules to the available facts.

Step 5: Display the inferred result.

Program

```
% Facts

bird(eagle).
bird(penguin).

has_wings(eagle).
has_wings(penguin).

can_fly(eagle).

cannot_fly(penguin).
```

```
% Forward Chaining Rules

animal(X) :-
    bird(X).

flying_bird(X) :-
    bird(X),
    has_wings(X),
    can_fly(X).

non_flying_bird(X) :-
    bird(X),
    cannot_fly(X).

% Inference Rule

infer(X) :-
    flying_bird(X).

infer(X) :-
    non_flying_bird(X).
```

Sample Input

```
?- infer(X).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl compiled, 32 lines read - 1748 bytes written, 8 ms

(16 ms) yes
| ?- infer(X).

X = eagle ? ;

X = penguin

(15 ms) yes
| ?-
```

Result

Hence, the Prolog program for **Forward Chaining** was executed successfully and the required facts were inferred.

Experiment 30

Title

Write a Prolog Program for Backward Chaining. Incorporate Required Queries.

Aim

To implement Backward Chaining in Prolog to determine whether a goal can be satisfied using the available facts and rules.

Objective

To understand how Prolog uses backward chaining to prove a goal by matching it with rules and facts in the knowledge base.

Algorithm

Step 1: Define the facts in the knowledge base.

Step 2: Define rules that relate the facts.

Step 3: Enter the goal through a query.

Step 4: Prolog searches the rules to satisfy the goal.

Step 5: Display the inferred result if the goal is proved.

Program

```
% Facts

bird(eagle).
bird(penguin).

has_wings(eagle).
has_wings(penguin).

can_fly(eagle).
cannot_fly(penguin).
```

```
% Backward Chaining Rules

flying_bird(X) :-
    bird(X),
    has_wings(X),
    can_fly(X).

non_flying_bird(X) :-
    bird(X),
    cannot_fly(X).

% Goal

goal(X) :-
    flying_bird(X).

goal(X) :-
    non_flying_bird(X).
```

Sample Input

```
?- goal(eagle).

?- goal(penguin).

?- goal(crow).
```

Sample Output

```
consult('C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl').
compiling C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl for byte code...
C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl compiled, 28 lines read - 1628 bytes written, 9 ms
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:3: redefining procedure bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:3: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:6: redefining procedure has_wings/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:6: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:9: redefining procedure can_fly/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:9: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:10: redefining procedure cannot_fly/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:11: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:14: redefining procedure flying_bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:18: previous definition
warning: C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp30.pl:19: redefining procedure non_flying_bird/1
         C:/Users/B SANTHOSH KUMAR/OneDrive/Documents/exp29.pl:23: previous definition

(63 ms) yes
| ?- goal(eagle).

true ?

(47 ms) yes
| ?- goal(penguin).

yes
| ?-
goal(crow).

no
| ?-
```

Result

Hence, the Prolog program for **Backward Chaining** was executed successfully and the required goal was proved.